

Defining Custom Convolution Kernels (Optional)

In some applications, it is useful to define custom convolution kernels manually instead of learning them through training. This is especially important for understanding classical image processing operations such as edge detection and gradient computation.

In this demo, horizontal and vertical Sobel filters are defined manually and can be used as fixed convolution kernels.

Custom Horizontal Filter (Sobel-X)

```
def my_Hfilter(shape, dtype=None):

    f = np.array([
        [[[-1]], [[0]], [[1]]],
        [[[-2]], [[0]], [[2]]],
        [[[-1]], [[0]], [[1]]]
    ])
    assert f.shape == shape
    return K.variable(f, dtype='float32')
```

Explanation (Line-by-Line):

- Defines a function to create a custom horizontal edge detection filter.
- The kernel corresponds to the Sobel-X operator.
- The filter highlights vertical edges in the image.
- The kernel values emphasize horizontal intensity changes.
- `assert f.shape == shape` ensures compatibility with the convolution layer.
- `K.variable` converts the NumPy array into a Keras backend tensor.

Custom Vertical Filter (Sobel-Y)

```
def my_Vfilter(shape, dtype=None):

    f = np.array([
        [[[-1]], [[-2]], [[-1]]],
        [[[0]], [[0]], [[0]]],
        [[[1]], [[2]], [[1]]]
    ])
    assert f.shape == shape
    return K.variable(f, dtype='float32')
```

```
assert f.shape == shape
return K.variable(f, dtype='float32')
```

Explanation (Line-by-Line):

- Defines a function to create a custom vertical edge detection filter.
- The kernel corresponds to the Sobel-Y operator.
- The filter highlights horizontal edges in the image.
- The kernel values emphasize vertical intensity changes.
- Ensures shape compatibility using the assert statement.
- Converts the kernel into a fixed tensor using the Keras backend.

Why Define Custom Kernels?

- To understand how convolution works at a low level.
- To apply classical image processing operators inside CNNs.
- To initialize convolution layers with known filters.
- To extract gradient-based features for denoising and edge-aware processing.
- To support hybrid CNN + traditional vision methods.

Using Custom Kernels in Conv2D

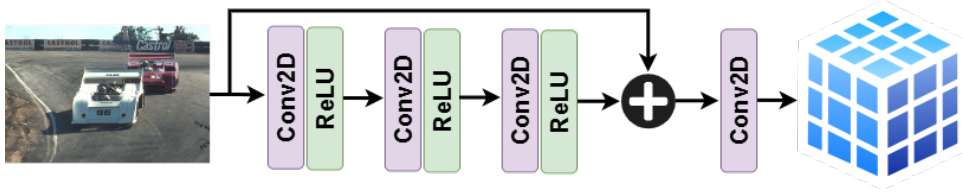
These custom filters can be used to initialize convolution layers as follows:

```
from tensorflow.keras import backend as K
from tensorflow.keras.layers import Conv2D

edge_x = Conv2D(1, (3, 3), padding='same',
                use_bias=False,
                kernel_initializer=my_Hfilter)

edge_y = Conv2D(1, (3, 3), padding='same',
                use_bias=False,
                kernel_initializer=my_Vfilter)
```

Explanation:



- Initializes Conv2D layers with fixed Sobel filters.
- `use_bias=False` ensures only convolution is applied.
- Enables gradient-based feature extraction inside CNN pipelines.

Demo: Residual CNN with Feature Map Visualization

This demo explains how to build a residual Convolutional Neural Network (CNN) corresponding to the given architecture diagram. The network consists of three stacked convolutional blocks followed by a residual addition and a final convolutional layer. The output feature maps of the last Conv2D layer are visualized for analysis.

Input and Preprocessing

The network takes a grayscale input image. The input image is normalized and reshaped to match the CNN input format.

- Input Type: Grayscale Image/Color Image
- Input Shape: $(None, None, 1)$ and $(None, None, 3)$
- Normalization: Pixel values scaled to $[0, 1]$

Python Code (Residual CNN + Visualization)

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

import tensorflow as tf
from tensorflow.keras.layers import Input, Conv2D, Activation, Add
from tensorflow.keras.models import Model
```

Explanation

- `numpy` is used for numerical array operations.
- `matplotlib` is used for visualizing feature maps.
- `cv2` (OpenCV) is used to load and resize the input image.
- TensorFlow and Keras modules are used to build the CNN model.

Image Loading and Preprocessing

```
img = cv2.imread("input_image.jpg", cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))
img = img.astype(np.float32) / 255.0

img_input = np.expand_dims(img, axis=(0, -1))
```

Line-by-Line Explanation

- `cv2.imread`: Loads the image in grayscale format.
- `cv2.resize`: Resizes the image to a fixed resolution.
- Conversion to `float32`: Required for neural network processing.
- Division by 255.0: Normalizes pixel values to the range $[0, 1]$.
- `np.expand_dims`: Adds batch and channel dimensions, resulting in shape $(1, H, W, 1)$.

CNN Model Construction

```
input_img = Input(shape=(None, None, 1))
```

Explanation: Defines the input tensor for a grayscale image with flexible height and width.

```
x1 = Conv2D(32, (3, 3), padding="same")(input_img)
x1 = Activation('relu')(x1)
```

Explanation: First convolutional block. Applies 32 filters of size 3×3 followed by ReLU activation to extract low-level features.

```
x2 = Conv2D(32, (3, 3), padding="same")(x1)
x2 = Activation('relu')(x2)
```

Explanation: Second convolutional block. Extracts mid-level features based on the output of the first block.

```
x3 = Conv2D(32, (3, 3), padding="same")(x2)
x3 = Activation('relu')(x3)
```

Explanation: Third convolutional block. Extracts deeper and more abstract features.

Residual (Skip) Connection

```
res = Add()([x1, x3])
```

Explanation:

- Implements the residual (+) connection shown in the diagram.
- Adds early-layer features (**x1**) to deep-layer features (**x3**).
- Helps gradient flow and reduces vanishing gradient problems.
- Preserves fine-grained spatial details.

Final Convolution Layer

```
final_conv = Conv2D(16, (3, 3), padding="same", name="final_conv")(res)
```

Explanation:

- Applies a final convolution after residual fusion.
- Reduces or transforms the number of feature channels.
- Produces the final deep feature cube for visualization.

Model Definition

```
model = Model(inputs=input_img, outputs=final_conv)
model.summary()
```

Explanation:

- Connects input and output tensors to form the full CNN model.
- `model.summary()` displays layer-wise details for verification.

Forward Pass and Feature Map Extraction

```
feature_maps = model.predict(img_input)
```

Explanation:

- Performs a forward pass through the CNN.
- Extracts the output of the final Conv2D layer.
- Stores the deep feature representations in `feature_maps`.

Feature Map Visualization

```
num_maps = feature_maps.shape[-1]
```

```
plt.figure(figsize=(12, 8))
```

```
for i in range(min(num_maps, 16)):
```

```
    plt.subplot(4, 4, i + 1)
```

```
    plt.imshow(feature_maps[0, :, :, i], cmap='gray')
```

```
    plt.title(f'FM {i+1}')
```

```
    plt.axis('off')
```

```
plt.suptitle("Output Feature Maps After Last Conv2D", fontsize=14)
```

```
plt.tight_layout()
```

```
plt.show()
```

Explanation

- Extracts the number of output channels.
- Creates a grid of subplots.
- Displays individual feature maps.
- Helps analyze what patterns the CNN has learned.

Educational Significance

- Demonstrates residual learning using skip connections.
- Helps visualize deep CNN representations.

- Improves understanding of hierarchical feature extraction.
- Supports viva questions on CNN internals and residual networks.

Conclusion

This demo provides a clear and practical understanding of how residual CNN architectures operate and how feature maps can be visualized to interpret learned representations. Students are encouraged to extend this model with additional convolutional blocks and experiment with different activation functions.